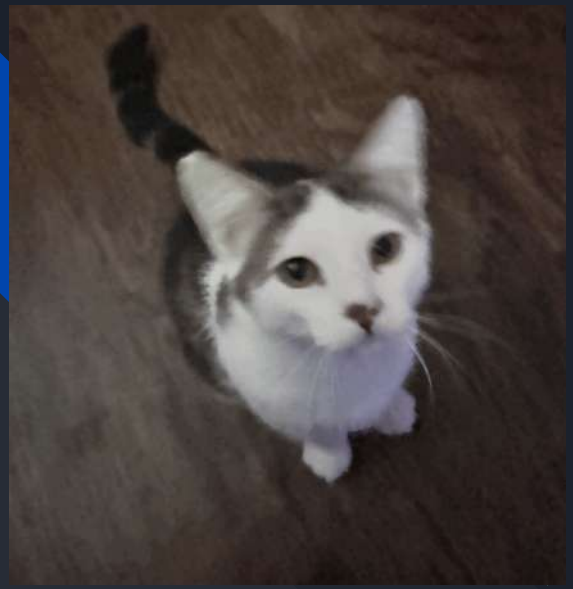


FundGoodDeeds

Team 3: JPY

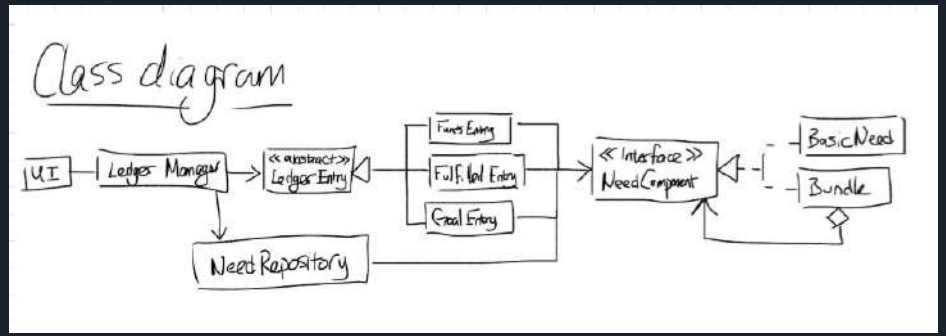
Jun Zheng, Evan Ke, and Kevin Zheng

Team Dynamics

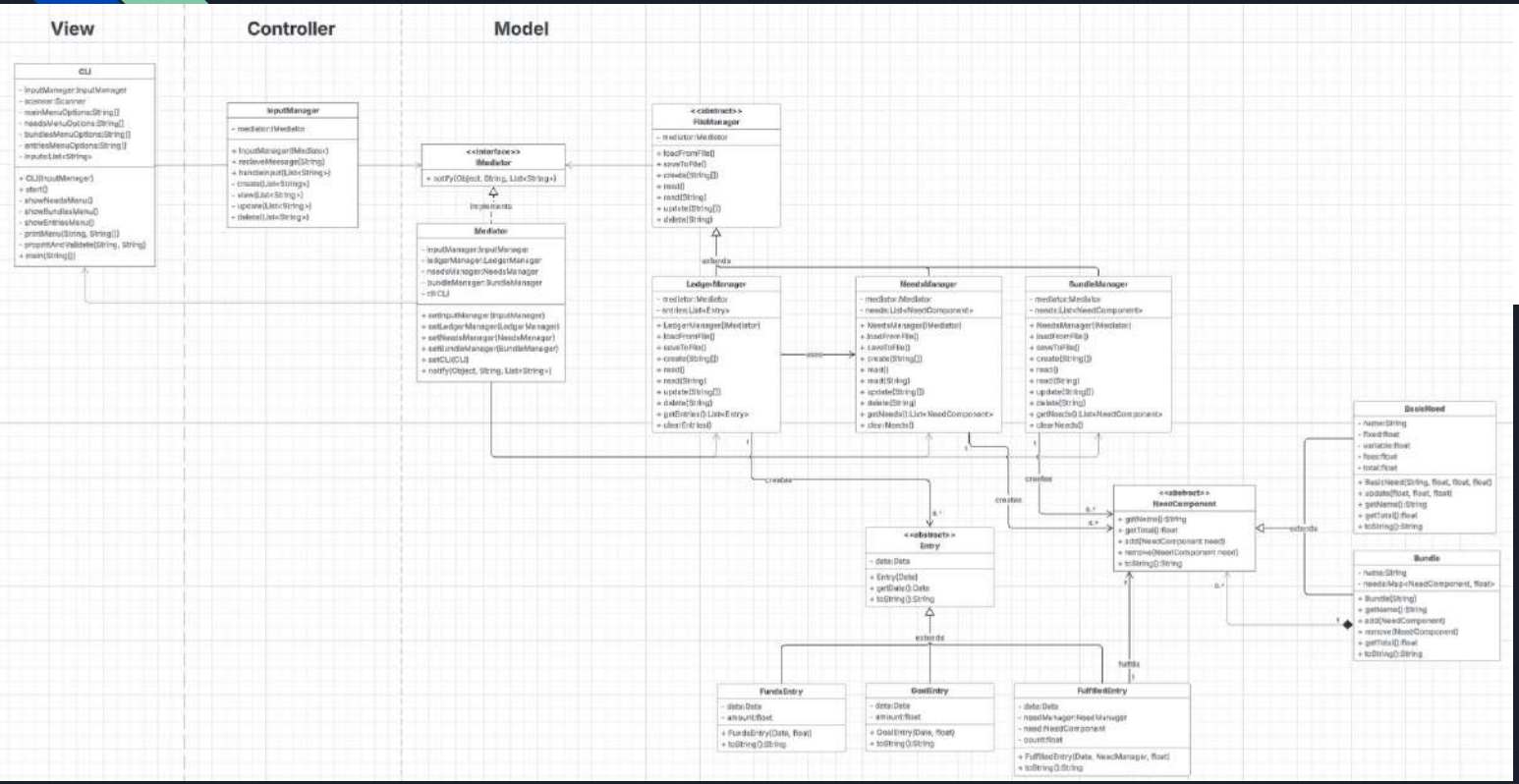
- Junheng Zheng, Team Coordinator
- Kevin Zheng, Configuration Manager, Design coordinator
- Evan Ke, Requirements coordinator, Quality coordinator

Design Process and Rationale

- Each team member created an initial version of the class diagram
- Combined individual diagrams to form the first collective version
- Refined diagram after receiving peer critiques and reviewing other groups' diagrams
- Made final refinements following actual implementation and testing
- Most of our work was done remotely due to our schedules.



Class Diagram

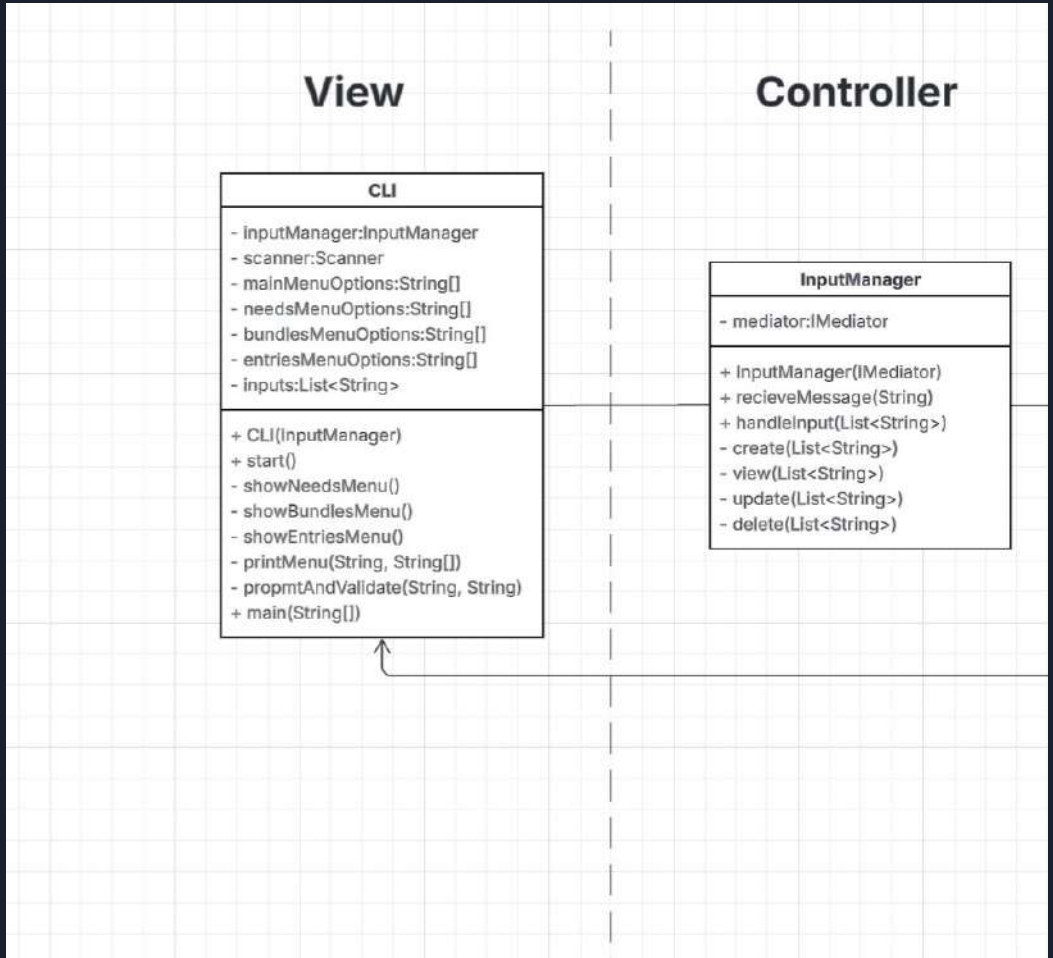


View: CLI

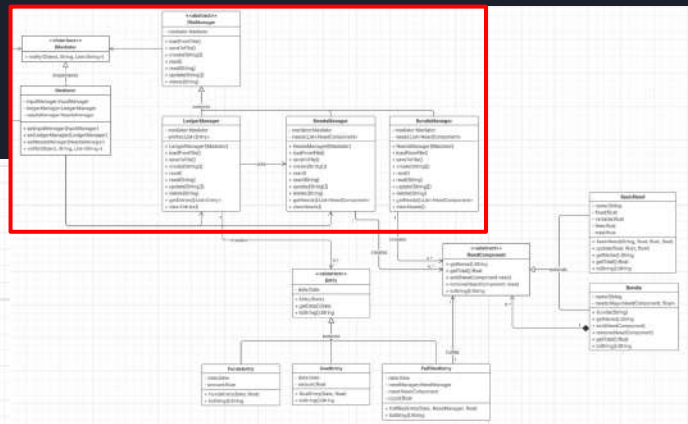
Controller: Input Manager

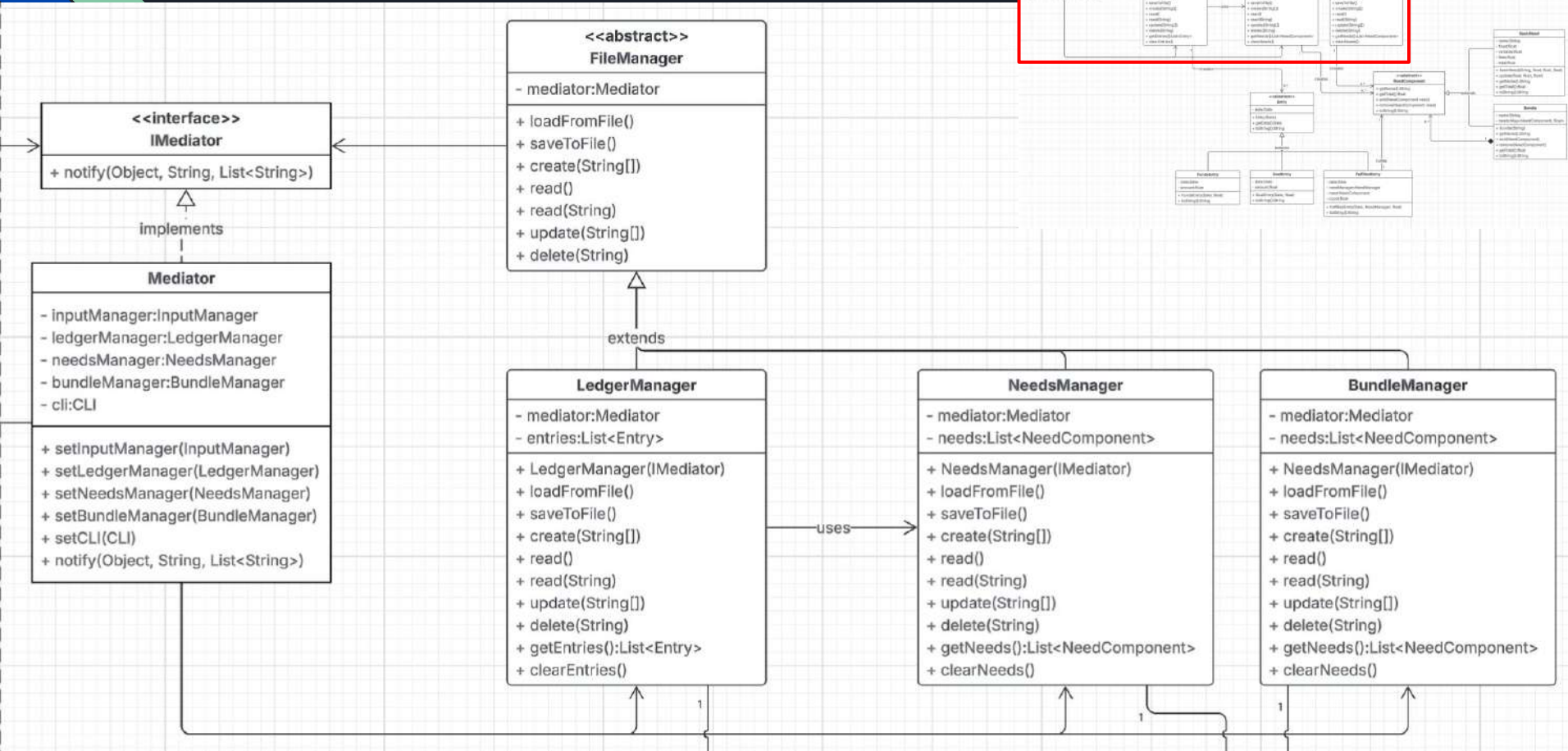
Model: EVERYTHING ELSE

View and Controller

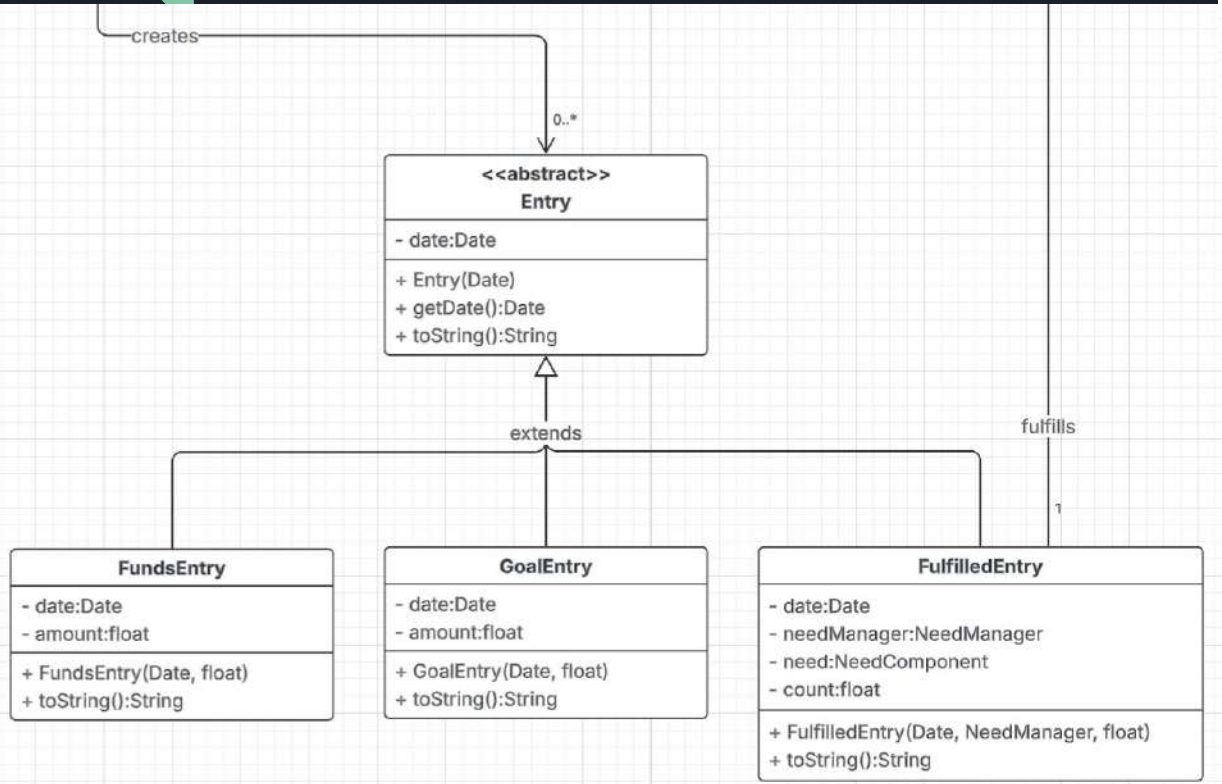


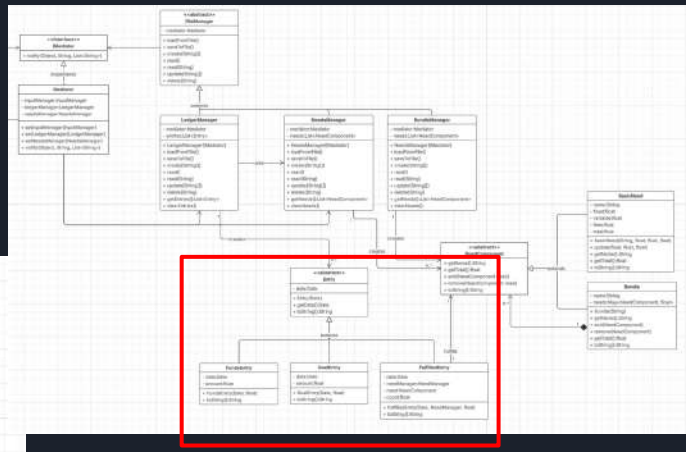
Model



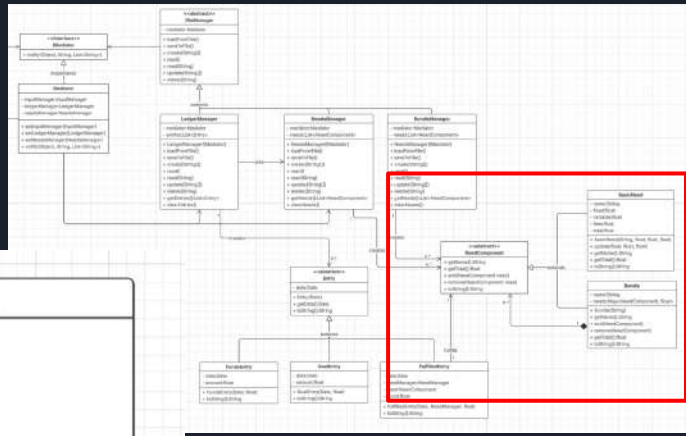

<<interface>>
IMediator
+ notify(Object, String, List<String>)
implements
Mediator
- inputManager:InputManager
- ledgerManager:LedgerManager
- needsManager:NeedsManager
- bundleManager:BundleManager
- cli:CLI
+ setInputManager(InputManager)
+ setLedgerManager(LedgerManager)
+ setNeedsManager(NeedsManager)
+ setBundleManager(BundleManager)
+ setCLI(CLI)
+ notify(Object, String, List<String>)
<<abstract>>
FileManager
- mediator:Mediator
+ loadFromFile()
+ saveToFile()
+ create(String[])
+ read()
+ read(String)
+ update(String[])
+ delete(String)
extends
LedgerManager
- mediator:Mediator
- entries:List<Entry>
+ LedgerManager(IMediator)
+ loadFromFile()
+ saveToFile()
+ create(String[])
+ read()
+ read(String)
+ update(String[])
+ delete(String)
+ getEntries():List<Entry>
+ clearEntries()
uses
NeedsManager
- mediator:Mediator
- needs:List<NeedComponent>
+ NeedsManager(IMediator)
+ loadFromFile()
+ saveToFile()
+ create(String[])
+ read()
+ read(String)
+ update(String[])
+ delete(String)
+ getNeeds():List<NeedComponent>
+ clearNeeds()
BundleManager
- mediator:Mediator
- needs:List<NeedComponent>
+ NeedsManager(IMediator)
+ loadFromFile()
+ saveToFile()
+ create(String[])
+ read()
+ read(String)
+ update(String[])
+ delete(String)
+ getNeeds():List<NeedComponent>
+ clearNeeds()
1
1
1

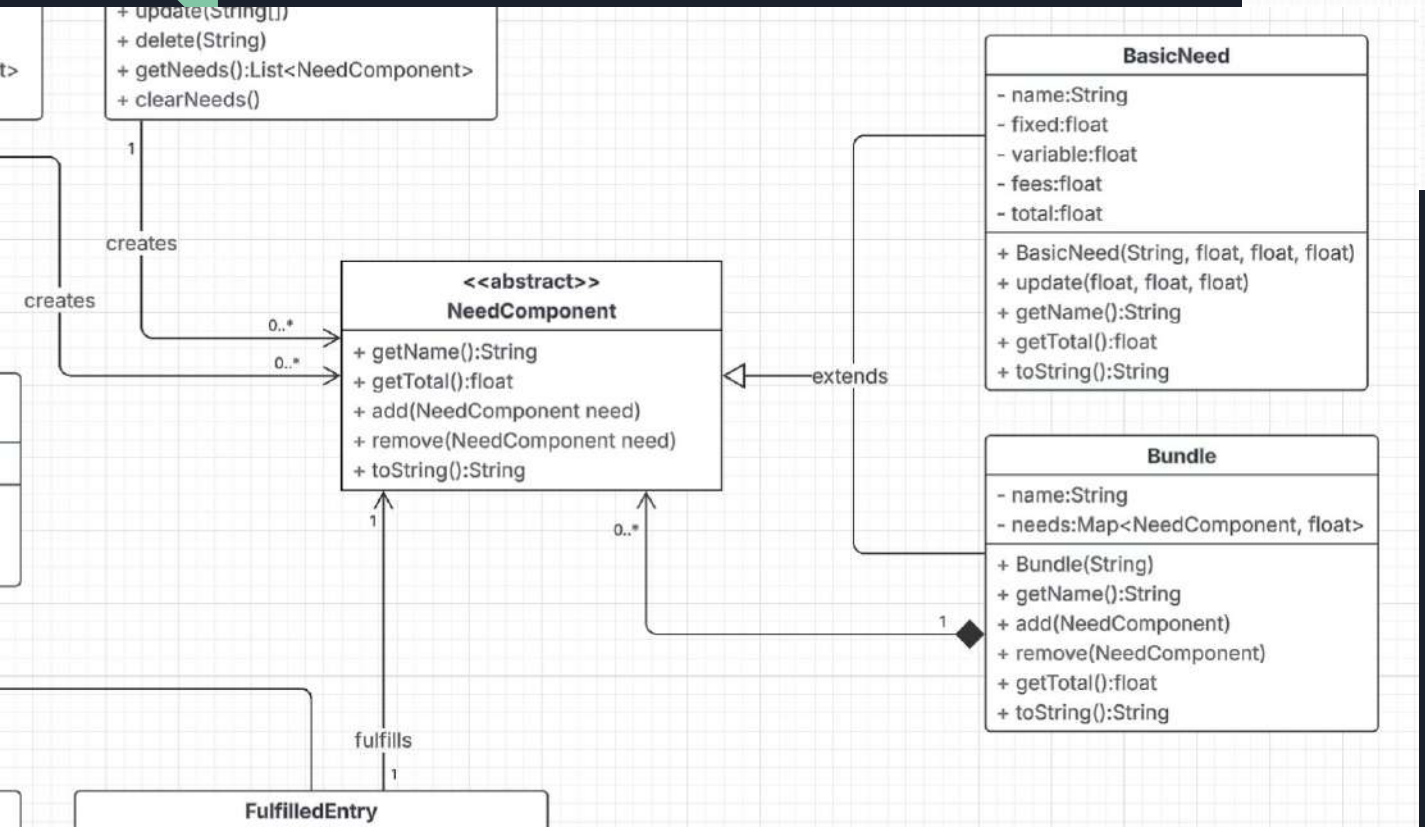
Model (contin.)





Model (contin.)

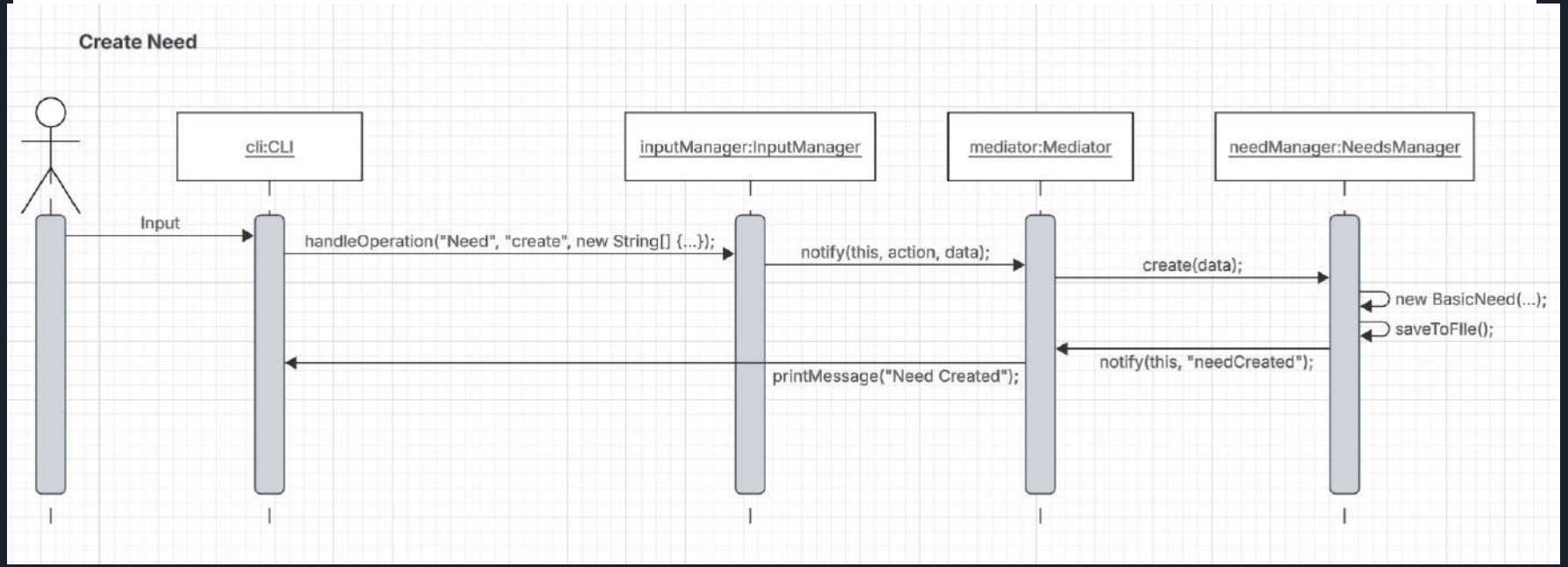




Sequence Diagrams

Create Need

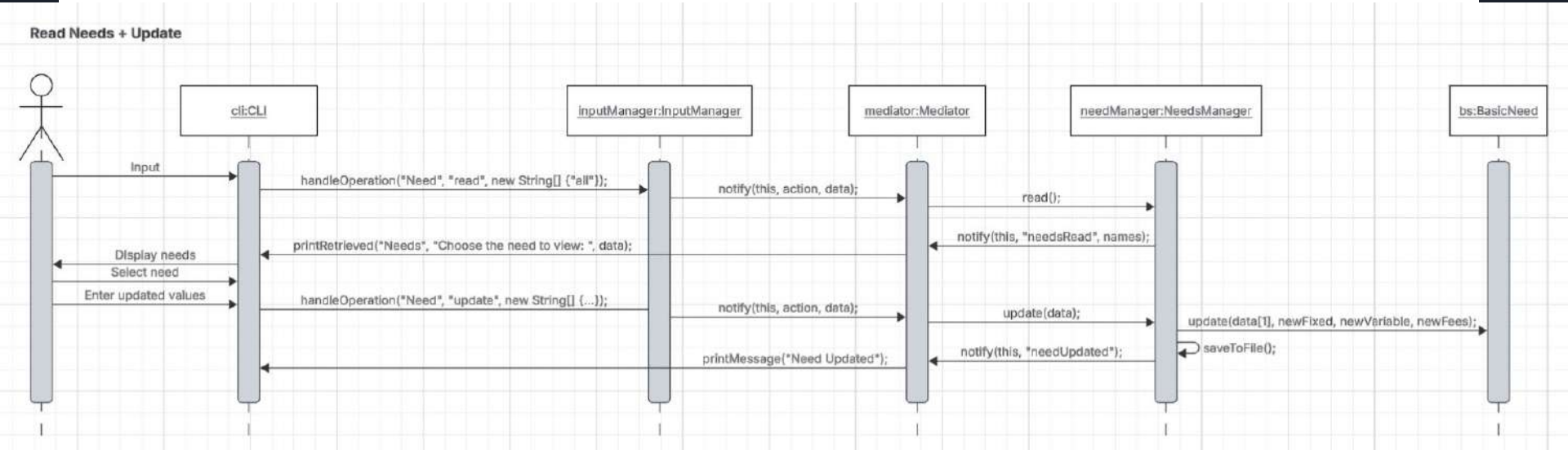
User inputs data into the View (CLI) -> CLI sends the data to the Controller (Input Manager) -> Input Manager notifies the Mediator about the request -> Mediator forwards the request to the Needs Manager -> Needs Manager creates and saves the new need, then notifies the Mediator _> Mediator updates the CLI (View) with the result.



Sequence Diagrams (contin.)

Read Needs + Update

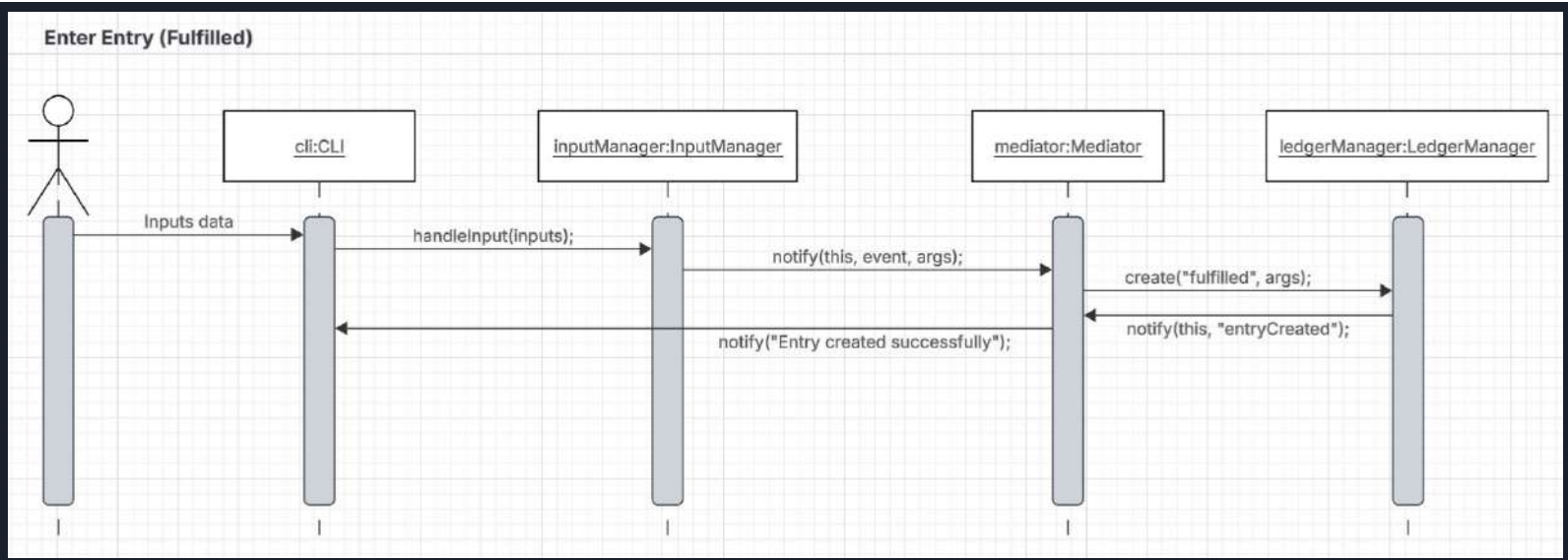
User Input -> CLI hands off input to Input Manager -> Input Manager notifies the Mediator -> Mediator makes read request to Needs Manager -> Needs Manager return the data by notifying the Mediator -> Mediator updates the CLI -> User makes selections and updates the values -> CLI sends the updated value to Input Manager -> Input Manager notifies the Mediator -> Mediator makes the update request -> Needs manager updates the need -> Needs Manager notifies the Mediator upon successful completion -> Mediator finally updates the CLI



Sequence Diagrams (contin.)

Enter Entry

An actor interacts with the CLI to initiate the creation of a fulfilled entry. The CLI sends the user input to the InputManager, which notifies the Mediator. The Mediator can request to display or add through the LedgerManager. If display, the mediator directs the Ledger Manager to display all entries available. If adding, the Mediator directs the Ledger Manager to create a new entry. On success, a confirmation message is returned to the CLI for display to the user.



Needs Functionality

Needs functionality handles everything related to community needs that requires support.

The Needs functionality this scope of the project

createNeed

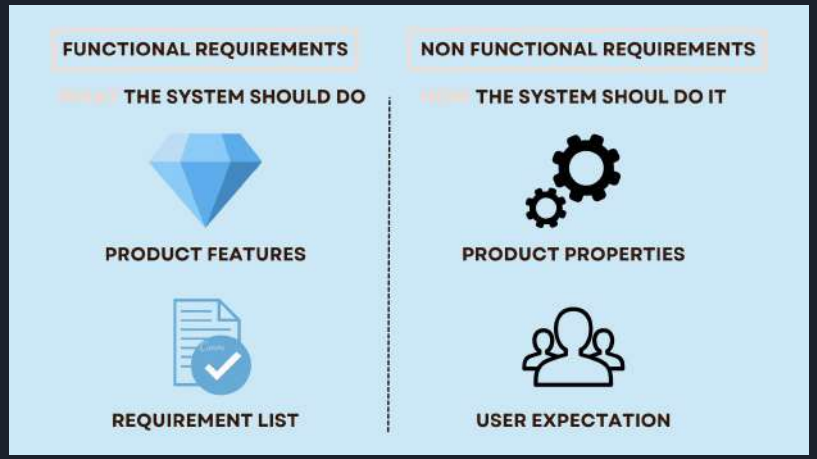
readNeed

updateNeed

deleteNeed

Where the user creates or views a need in the CLI, the Mediator sends that request to the Needs.

This functionality is responsible for adding, viewing, editing, and removing needs, in addition to saving and loading from a file. This part of our program keeps track of all the open or fulfilled needs that our system manages.



Bundles Functionality

The BundleManager in our program manages commands like:

createBundle

readBundle

updateBundle

deleteBundle.

When an user creates or modifies a bundle through the CLI, the Mediator routes that command to the BundleManager.

It saves bundle information to a file, and loads it when the program starts up. This part of our program manages resources sent out to Needs.

Entry Functionality

The entry serves as a central logging system for the app, allowing users to add goals, manage funds, and record fulfilled achievements to keep track of their overall progress.

We followed phase 1 deliverables, which included:

Adding entries, viewing entries, saving entries, and loading entries.

In the future, we can also implement functionality outside of scope, such as:

Updating and deleting functionality.

```
-----------------------------
ENTRIES MENU
1. Create New Entry
2. View Entries
3. Back
-----------------------------
Choose an option: 2
[CONTROLLER] Read-ing entry...
[CONTROLLER] notifying the mediator...readEntry
[MEDIATOR] Routing ReadEntries to LedgerManager
[Model] LedgerManager: Reading entries...
--------ALL AVAILABLE ENTRIES--------

  1                 2025,4,1,f,200.00
  2                 2025,4,1,g,2000.00
  3                 2025,4,2,f,180.50
  4                 2025,4,3,g,1800.00
  5                 2025,4,3,f,150.00
  6                 2025,4,5,f,300.00
  7                 2025,4,5,g,2500.00
  8                 2025,4,4,f,20.00

-------------------------------------
[Model] LedgerManager: Finished reading entries.
[MEDIATOR] Notifying InputManager of EntriesRead
[CONTROLLER] InputManager received: Displaying Entries!
-----------------------------
```

```
-----------------------------
ENTRIES MENU
1. Create New Entry
2. View Entries
3. Back
-----------------------------
Choose an option: 1
Enter entry type (funds/goal/fulfilled): funds
Enter date (yyyy-mm-dd): 2025-04-04
Enter amount: 20
[CONTROLLER] Create-ing entry...
[CONTROLLER] notifying the mediator...createEntry
[MEDIATOR] Routing CreateEntry to LedgerManager
[Model] LedgerManager: Creating entry...
[Model] LedgerManager: FundsEntry created.
2025,4,4,f,20.00
[Model] LedgerManager: Saving entries to file...
[MEDIATOR] Notifying InputManager of EntryCreated
[CONTROLLER] InputManager received: Entry created successfully!
-----------------------------
ENTRIES MENU
1. Create New Entry
2. View Entries
3. Back
-----------------------------
```

Challenges

Challenges

Working through assignment ambiguities.

Figuring out how we should split the work given our constraints. This included:

- Time Limit
- Small Group Size
- Learning about new constraints

What We Learned & Successes

Collaboration

We had to work together in order to get all the functionality done. We each did one part, and merged the changes later on once we finished.

Utilizing Design Patterns

Throughout the project, we had to use design patterns such as the composite and MVC pattern. Learning about these patterns helped us get better at understanding why design patterns are necessary to making scalable and reliable software.

Time Management

This phase of the project had many deadlines and deliverables. We had to collaborate and work together to ensure that we planned and worked on the assignment on a timely matter.

DEMO!